

RAPPORT DE TRAVAUX PRATIQUES

Integration Continue et Livraison Continue avec Jenkins

FRENDI Belkacem

Depot GitHub : <https://github.com/frebelkacem-cell/devops-jenkins-tp>

9 travaux pratiques • Jenkins, Docker, Maven, GitHub, SonarQube, Tomcat

Contexte general

Dans le cadre de ce cours consacré à Jenkins et à l'intégration/livraison continue, j'ai mis en place un environnement de travail complet avant de commencer les TP. Plutôt que d'installer Jenkins directement sur mon poste, j'ai choisi de le faire tourner dans un conteneur Docker (image officielle `jenkins/jenkins:lts-jdk21`), ce qui permet de garder l'installation isolée et facile à réinitialiser si besoin.

Jenkins est accessible depuis mon navigateur sur `http://localhost:8081` (le port 8080 par défaut était déjà utilisé par un autre conteneur qui tournait déjà sur ma machine, un stack Traefik sans rapport avec ce cours). Les données de Jenkins (jobs, configuration, plugins) sont conservées dans un volume Docker nommé `jenkins_home`, ce qui permet de recréer le conteneur sans perdre le travail déjà effectué.

Au fil des TP, j'ai eu besoin d'ajouter d'autres services (SonarQube, Tomcat, un agent Jenkins) qui devaient pouvoir communiquer entre eux par leur nom de conteneur. Le réseau Docker par défaut ne permettant pas cette résolution de noms, j'ai créé un réseau bridge dédié (`devops-net`) et j'y ai connecté tous les conteneurs du projet.

Pour éviter de recréer un projet différent à chaque TP, j'ai choisi de développer un seul petit projet Java (Maven), une calculatrice simple (opérations d'addition, soustraction, multiplication et division), réutilisée sur l'ensemble des TP du cours. Ce projet est hébergé sur GitHub à l'adresse suivante :

<https://github.com/frebelkacem-cell/devops-jenkins-tp>

Cette approche présente l'avantage de montrer une vraie continuité de pipeline : le même code source est compilé, testé, analysé, packagé puis déployé au fil des TP, avec un `pom.xml` qui évolue progressivement (ajout de JaCoCo, passage en packaging WAR, etc.), comme cela se ferait dans un vrai projet DevOps.

TP1 - Creation et compilation d'un projet Freestyle

Objectif

Créer un premier job Jenkins et compiler automatiquement le projet à chaque build.

Demarche

J'ai créé un nouveau job de type «Projet free-style» nommé TP1-compilation belkacem. Dans la section «Gestion de code source», j'ai configuré Jenkins pour qu'il aille chercher le code directement sur mon dépôt GitHub (branche main).

Pour l'étape de build, j'ai d'abord voulu compiler directement une classe isolée, mais je me suis rendu compte que cela ne correspondait pas à la structure réelle de mon projet Maven : je suis donc revenu à une étape «Exécuter un script shell» plus simple et plus représentative, avec la commande `mvn compile`, qui s'appuie sur le `pom.xml` du projet.

```
mvn compile
```

Resultats obtenus

Le premier build (#1) s'est terminé avec succès (statut vert) : le clone du dépôt Git s'est bien déroulé et Maven a compilé le projet sans erreur.

Le deuxième build (#2) est en revanche passé en échec : j'avais laissé une erreur dans la configuration du job pendant que je testais différentes options. La console m'a clairement indiqué le point de blocage, ce qui m'a permis de corriger rapidement la configuration avant de relancer un build.

Jenkins / TP1-compilation belkacem

État

TP1-compilation belkacem

Liens permanents

- Dernier build (#1), il y a 4 mn 41 s
- Dernier build stable (#1), il y a 4 mn 41 s
- Dernier build avec succès (#1), il y a 4 mn 41 s
- Dernier build complété (#1), il y a 4 mn 41 s

Builds >

Filter

Today

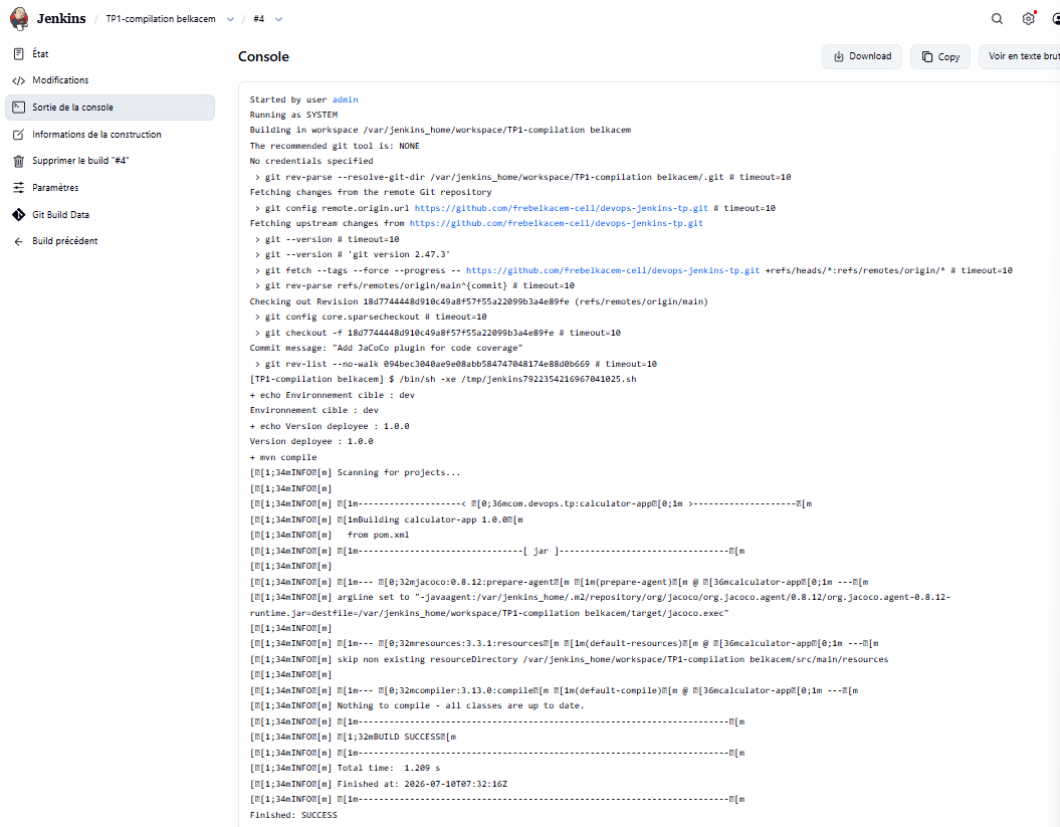
- #2 09:03
- #1 08:58

Historique des builds : #1 réussi (vert), #2 en échec (rouge)

Une fois la configuration corrigée, j'ai relancé le job à plusieurs reprises au fil de l'avancement du projet (par exemple après l'ajout du plugin JaCoCo dans le pom.xml), et la compilation s'est à chaque fois terminée par BUILD SUCCESS.



Historique complet du job TP1-compilation belkacem



Sortie console d'un build réussi : checkout Git puis mvn compile (BUILD SUCCESS)

Livrable

LIVRABLE

Job Freestyle fonctionnel, avec un exemple reel de build en echec correctement diagnostique via la console, puis corrige.

TP2 - Creation et compilation d'un projet Maven

Objectif

Industrialiser le build avec Maven et publier automatiquement les resultats de tests.

Demarche

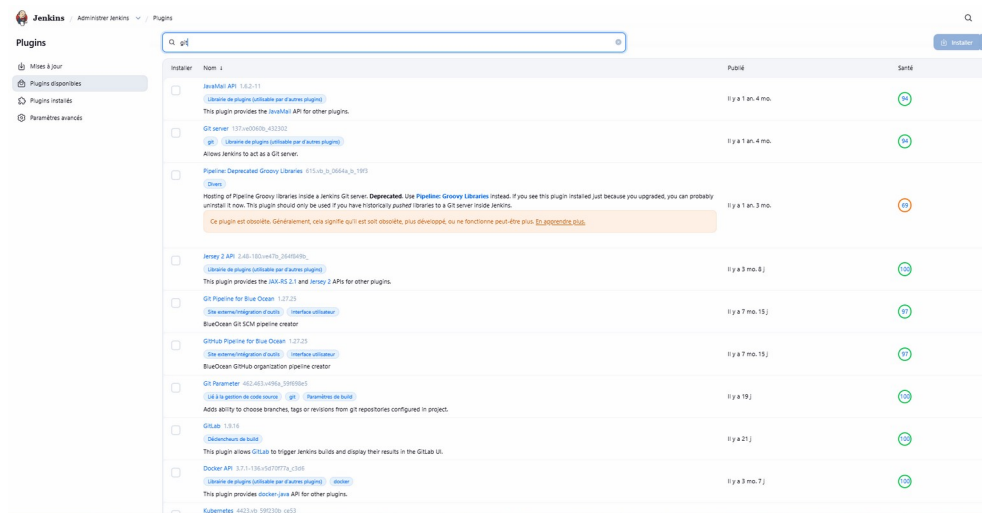
Pour ce TP, j'ai reutilise le meme depot GitHub, cette fois avec le projet Maven complet (pom.xml, classes principales et tests unitaires JUnit 5 dans CalculatorTest).

J'ai cree un nouveau job de type «Construire un projet Maven» (TP2-maven-project), pointe vers le meme depot Git (branche main), avec le pom.xml a la racine et l'objectif Maven suivant :

```
clean install
```

Resultats obtenus

Le build a execute l'ensemble du cycle de vie Maven : clean, compile, test, package puis install. Les 5 tests unitaires de la classe CalculatorTest (addition, soustraction, multiplication, division et division par zero) se sont tous executes avec succes, et Jenkins a publie automatiquement le rapport de tests JUnit/Surefire correspondant : 0 echec sur 5 tests, avec le detail par module (com.devops.tp:calculator-app).



Installer	Nom	Publié	Santé
☐	jenkins API 1.6.2-11 Librairie de plugins (utilisable par d'autres plugins) This plugin provides the Jenkins API for other plugins.	Il y a 1 an, 4 mo.	OK
☐	Git server 1371e00079_432332 Librairie de plugins (utilisable par d'autres plugins) Allows Jenkins to act as a Git server.	Il y a 1 an, 4 mo.	OK
☐	Pipeline: Groovy Libraries 0154b3_2004a3_1003 Librairie de plugins (utilisable par d'autres plugins) Hosting of Pipeline Groovy libraries inside a Jenkins Git server. Deprecated: Use Pipeline: Groovy Libraries instead. If you see this plugin installed just because you upgraded, you can probably uninstall it now. This plugin should only be used if you have historically authored libraries to a Git server inside Jenkins. Ce plugin est obsolète. Généralement, cela signifie qu'il est soit obsolète, plus développé, ou ne fonctionne peut-être plus. En savoir plus.	Il y a 1 an, 3 mo.	Warning
☐	Jersey 2 API 2.40-102-e470_20450490 Librairie de plugins (utilisable par d'autres plugins) This plugin provides the JAX-WS 2.1 and Jersey 2 APIs for other plugins.	Il y a 3 mo, 8 j.	OK
☐	Git Pipeline for Blue Ocean 1.27.23 Voir l'extension d'outils Créer une utilisation BlueOcean-Git SCM pipeline creator	Il y a 7 mo, 15 j.	OK
☐	GitHub Pipeline for Blue Ocean 1.27.23 Voir l'extension d'outils Créer une utilisation BlueOcean-GitHub pipeline creator	Il y a 7 mo, 15 j.	OK
☐	Git Parameter 4024d3_4090a_59105643 Librairie de plugins (utilisable par d'autres plugins) Documentation de base Adds ability to choose branches, tags or revisions from git repositories configured in project.	Il y a 19 j.	OK
☐	GitLab 1.0.14 Documentation de base This plugin allows GitLab to trigger Jenkins builds and display their results in the GitLab UI.	Il y a 21 j.	OK
☐	Docker API 3.7.1-130-e507077a_1106 Librairie de plugins (utilisable par d'autres plugins) docker This plugin provides docker Java API for other plugins.	Il y a 3 mo, 7 j.	OK

Page "Resultat des tests" du job TP2-maven-project : 5 tests, 0 echec

En reprenant l'historique du job un peu plus tard (apres avoir integre le webhook GitHub, cf. TP3), on voit bien la mecanique complete : un premier build vert, puis un deuxieme build declenche automatiquement qui est passe en rouge suite au bug d'encodage explique dans la section suivante, avec le graphique de tendance des tests qui suit cette evolution.



Historique du job Maven avec la courbe de tendance des tests

Livrable

LIVRABLE

Job Maven fonctionnel exécutant le cycle complet clean/compile/test/package/install, avec publication automatique et consultable du rapport de tests JUnit.

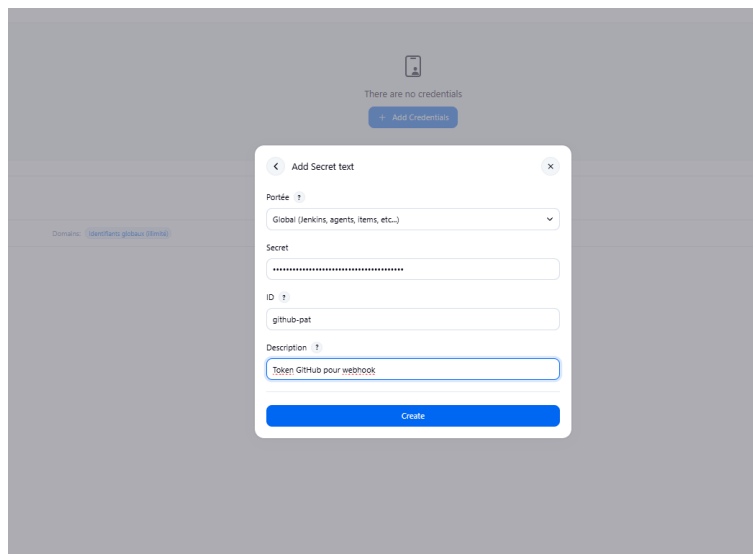
TP3 - Interconnexion Jenkins - GitHub (webhook)

Objectif

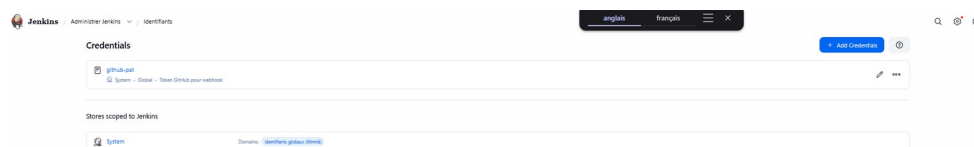
Declencher automatiquement un build Jenkins a chaque push sur GitHub.

Demarche

Le plugin «GitHub» etait deja present parmi les plugins installes par default sur mon instance Jenkins. J'ai genere un token d'accès personnel GitHub (PAT, classic) avec les scopes repo et admin:repo_hook, puis je l'ai declare dans Jenkins sous la forme d'un credential de type «Secret text» (ID : github-pat).

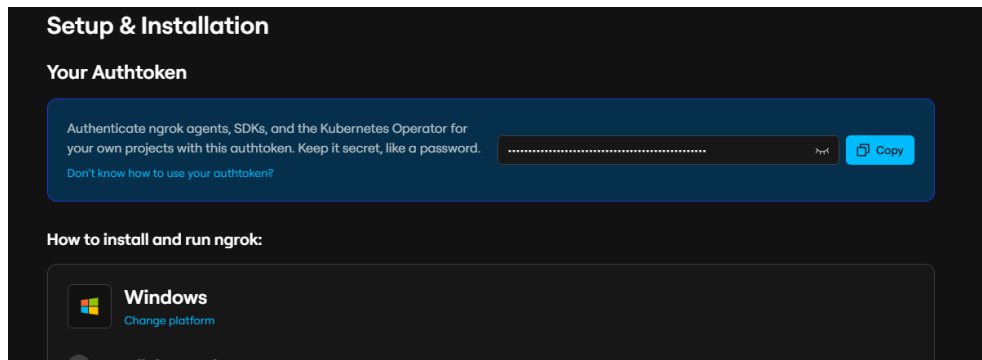


Ajout du credential github-pat dans Jenkins



Le credential github-pat apparait bien dans la liste des identifiants

Le point delicat de ce TP est que mon Jenkins tourne uniquement en local (http://localhost:8081) : GitHub, sur Internet, ne peut pas l'atteindre directement. J'ai donc utilise ngrok pour ouvrir un tunnel public temporaire vers mon Jenkins local, ce qui m'a fourni une URL publique en https://xxxx.ngrok-free.dev redirigeant vers http://localhost:8081.

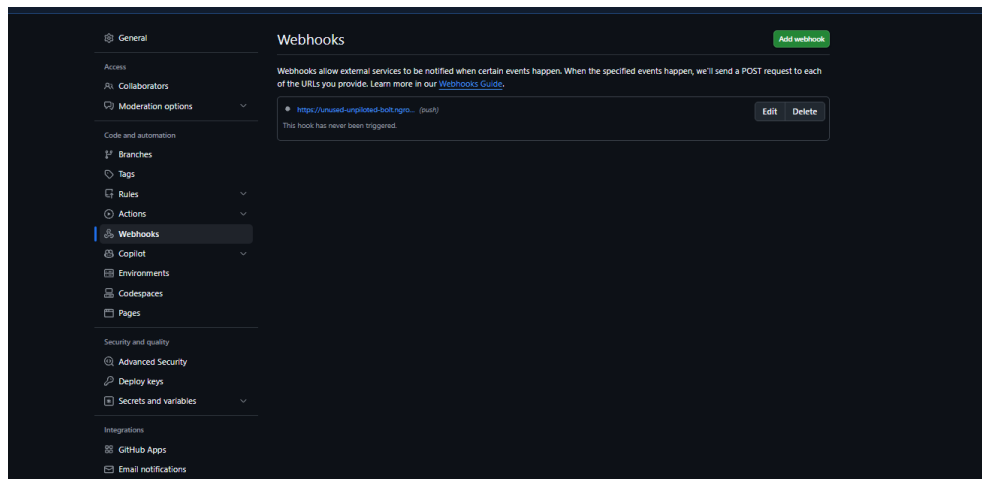


Tunnel ngrok actif, redirigeant vers localhost:8081

J'ai ensuite cree un webhook sur le depot GitHub (Settings > Webhooks > Add webhook) avec comme Payload URL l'adresse ngrok suivie de /github-webhook/, en format application/json, declenche sur l'evenement push. Enfin, dans la configuration du job Jenkins, j'ai active le declencheur «GitHub hook trigger for GITScm polling».

Resultats obtenus

Le ping de test envoye automatiquement par GitHub lors de la creation du webhook a reussi, confirmant que Jenkins etait bien joignable depuis Internet via le tunnel ngrok.



Onglet Recent Deliveries du webhook GitHub : ping envoye avec succes

J'ai ensuite teste le declenchement reel en modifiant un fichier du projet directement depuis PowerShell et en poussant le commit. Un nouveau build s'est lance automatiquement dans Jenkins, sans aucune action manuelle de ma part : le webhook fonctionnait bien de bout en bout.

Ce build s'est toutefois solde par un echec, non provoque volontairement cette fois : la commande PowerShell que j'avais utilisee pour modifier le fichier (echo ... >> App.java) a encode le texte ajoute en UTF-16 alors que le fichier source est en UTF-8, ce qui a introduit des

caracteres nuls dans le code. La console Maven affichait des erreurs de compilation du type «illegal character '\u0000'».

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Tous droits réservés.

Installez la dernière version de PowerShell pour de nouvelles fonctionnalités et améliorations ! https://aka.ms/PSWindows

PS C:\Users\belka> cd C:\Users\belka\Downloads\DevOps\devops-tp-project
PS C:\Users\belka\Downloads\DevOps\devops-tp-project> echo "// test webhook" >> src\main\java\com\devops\tp\App.java
PS C:\Users\belka\Downloads\DevOps\devops-tp-project> git add .
PS C:\Users\belka\Downloads\DevOps\devops-tp-project> git commit -m "test webhook"
[main b92cad7] test webhook
1 file changed, 0 insertions(+), 0 deletions(-)
PS C:\Users\belka\Downloads\DevOps\devops-tp-project> git push
Enumerating objects: 17, done.
Counting objects: 100% (17/17), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (9/9), 631 bytes | 315.00 KiB/s, done.
Total 9 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/frebelkacem-cell/devops-jenkins-tp.git
094bec3..b92cad7 main -> main
PS C:\Users\belka\Downloads\DevOps\devops-tp-project> |
```

PowerShell : commit/push ayant declenche le build en echec (probleme d'encodage)

Après avoir identifié la cause, j'ai restauré le fichier à son état précédent avec git checkout <commit> -- App.java, puis commit et push. Ce nouveau push a de nouveau déclenché automatiquement un build, qui s'est cette fois terminé par BUILD SUCCESS.

```
PS C:\Users\belka\Downloads\DevOps\devops-tp-project> cd C:\Users\belka\Downloads\DevOps\devops-tp-project
PS C:\Users\belka\Downloads\DevOps\devops-tp-project> git checkout 094bec3040ae9e08abb584747048174e88d0b669 -- src\main\
java\com\devops\tp\App.java
PS C:\Users\belka\Downloads\DevOps\devops-tp-project> git add .
PS C:\Users\belka\Downloads\DevOps\devops-tp-project> git commit -m "fix: revert encoding issue introduced by PowerShell
echo"
[main 295b522] fix: revert encoding issue introduced by PowerShell echo
1 file changed, 0 insertions(+), 0 deletions(-)
PS C:\Users\belka\Downloads\DevOps\devops-tp-project> git push
Enumerating objects: 17, done.
Counting objects: 100% (17/17), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (9/9), 635 bytes | 317.00 KiB/s, done.
Total 9 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/frebelkacem-cell/devops-jenkins-tp.git
b92cad7..295b522 main -> main
PS C:\Users\belka\Downloads\DevOps\devops-tp-project> |
```

PowerShell : restauration du fichier via git checkout, puis commit/push du correctif

Livable

LIVRABLE

Webhook GitHub -> Jenkins fonctionnel : chaque push déclenche automatiquement un build, avec un exemple réel d'échec (problème d'encodage) correctement diagnostiqué et corrigé.

TP4 - Qualite de code avec SonarQube et JaCoCo

Objectif

Mettre en place une analyse automatique de la qualite du code (bugs, duplication, couverture de tests) a chaque build, avec un Quality Gate.

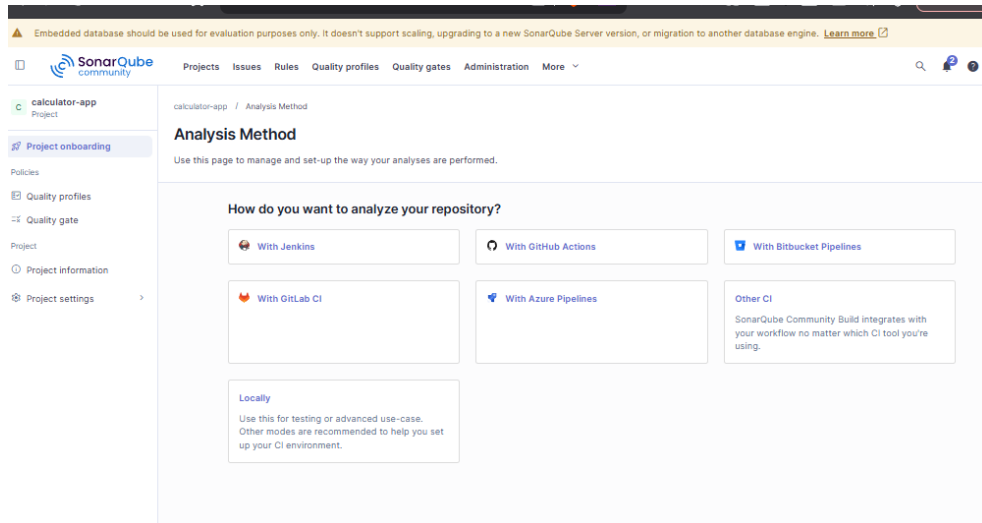
Demarche

J'ai demarre un conteneur SonarQube Community Edition, connecte au meme reseau Docker devops-net que Jenkins, afin que Jenkins puisse joindre SonarQube par son nom de conteneur (<http://sonarqube:9000>).

Dans le pom.xml du projet, j'ai ajoute le plugin jacoco-maven-plugin, avec une execution prepare-agent (pour instrumenter les tests) et une execution report (pour generer le rapport de couverture apres les tests).

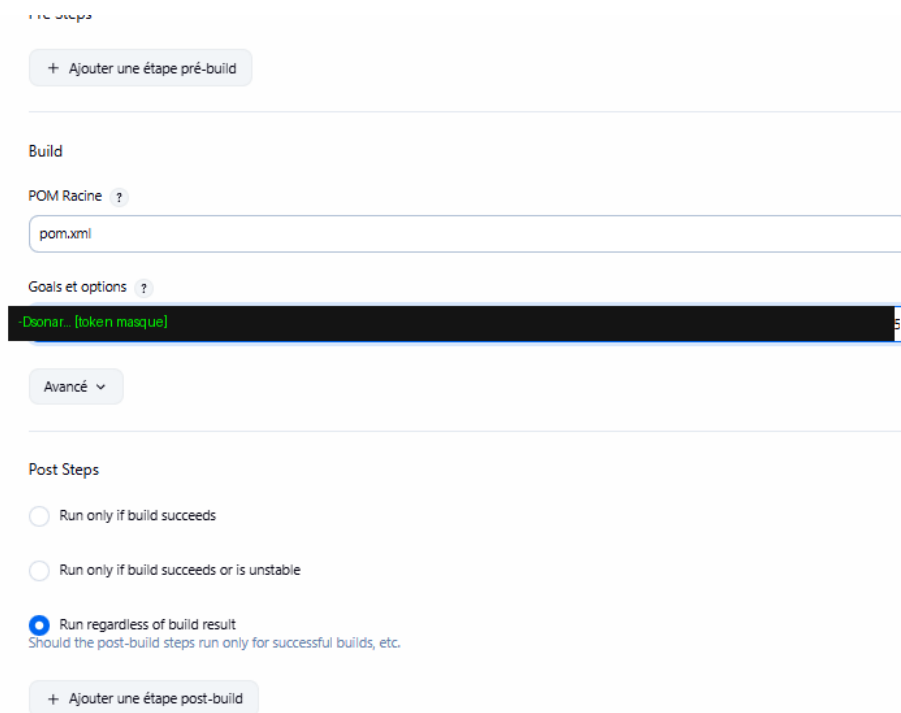
```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.12</version>
  <executions>
    <execution><id>prepare-agent</id><goals><goal>prepare-agent</goal></goals></execution>
    <execution><id>report</id><phase>test</phase><goals><goal>report</goal></goals></execution>
  </executions>
</plugin>
```

Dans SonarQube, j'ai cree le projet calculator-app et choisi la methode d'analyse «Locally» (execution depuis Jenkins avec un token), plutot que les integrations toutes faites proposees (GitHub Actions, GitLab CI...) qui ne correspondaient pas a mon cas.



SonarQube - choix de la methode d'analyse (Locally)

J'ai genere un token d'analyse, puis ajoute une etape de build supplementaire dans le job Jenkins avec les objectifs Maven suivants (le token a ete masque sur la capture ci-dessous pour la publication de ce rapport) :



Etape de build Jenkins avec les proprietes sonar.projectKey / sonar.host.url / sonar.token

Resultats obtenus

Au tout premier lancement, l'analyse SonarQube affichait bien le Quality Gate passe, mais la couverture de code restait a 0.0% alors que j'avais bien ajoute JaCoCo au pom.xml. En cherchant

l'origine du probleme, je me suis rendu compte que j'avais oublie de commiter et pousser cette modification du pom.xml avant de relancer le build : Jenkins recuperait donc encore l'ancienne version du fichier depuis GitHub.

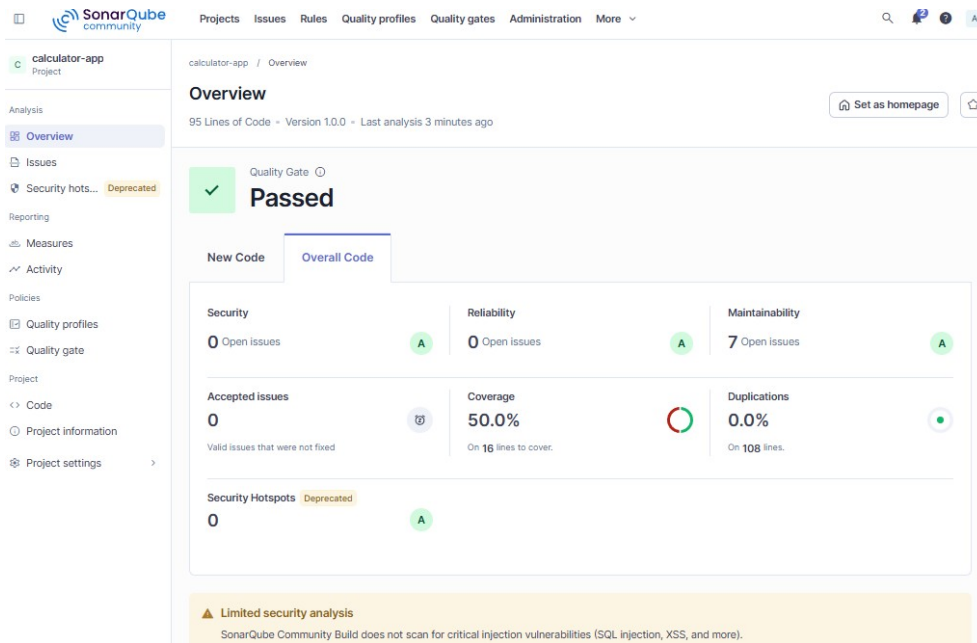
The screenshot displays the SonarQube Community interface for the 'calculator-app' project. The 'Overview' page shows a 'Passed' Quality Gate. The 'Overall Code' tab is active, displaying various metrics:

Metric	Value	Status
Security	0 Open issues	Passed (A)
Reliability	0 Open issues	Passed (A)
Maintainability	7 Open issues	Passed (A)
Accepted issues	0	Passed (A)
Coverage	0.0% (On 14 lines to cover)	Failed (Red circle)
Duplications	0.0% (On 88 lines)	Passed (Green circle)
Security Hotspots	0	Passed (A)

A warning message at the bottom states: 'Limited security analysis. SonarQube Community Build does not scan for critical injection vulnerabilities (SQL injection, XSS, and more). Explore other editions'.

Premiere analyse : couverture a 0.0% (pom.xml pas encore pousse sur GitHub)

Apres avoir commite et pousse le pom.xml a jour puis relance le build, la couverture est bien remontee a 50.0%, avec le Quality Gate toujours au vert.



Après correction : couverture a 50.0%, Quality Gate Passed

Livable

LIVRABLE

Analyse SonarQube automatisée depuis Jenkins, avec couverture de code JaCoCo remontée correctement et Quality Gate passe, après diagnostic d'un oubli de commit/push.

TP5 - Projet parametre (build parameters)

Objectif

Rendre un job Jenkins configurable au moment du lancement, via des parametres saisis par l'utilisateur (par exemple l'environnement cible et la version a deployer).

Demarche

J'ai repris le job de compilation et coche l'option «Ce build a des parametres», puis ajoute deux parametres de type chaine de caracteres : un pour l'environnement cible (ENVIRONNEMENT, valeur par default dev) et un pour la version deployee (VERSION, valeur par default 1.0.0). Dans le script de build, j'ai ensuite reference ces parametres comme des variables d'environnement classiques.

```
echo "Environnement cible : $ENVIRONNEMENT"  
echo "Version deployee : $VERSION"  
java -version && javac -version  
mvn compile
```

Resultats obtenus

Page 16 / 31

Environnement cible : dev
Version deployee : 1.0.0

Ce mecanisme est interessant car il permet, avec un seul job, de simuler differents contextes de deploiement (dev, recette, prod) simplement en changeant la valeur des parametres au moment du lancement, sans dupliquer la configuration.

Livrable

LIVRABLE

Job Jenkins parametre fonctionnel, dont le comportement (variables affichees dans la console) s'adapte aux valeurs saisies au lancement du build.

TP6 - Deploiement automatise sur Tomcat

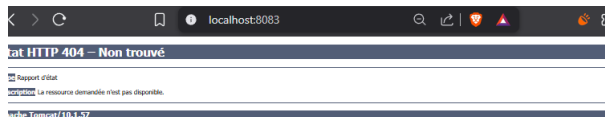
Objectif

Empaqueter l'application au format WAR et la deployer automatiquement sur un serveur Tomcat via Jenkins.

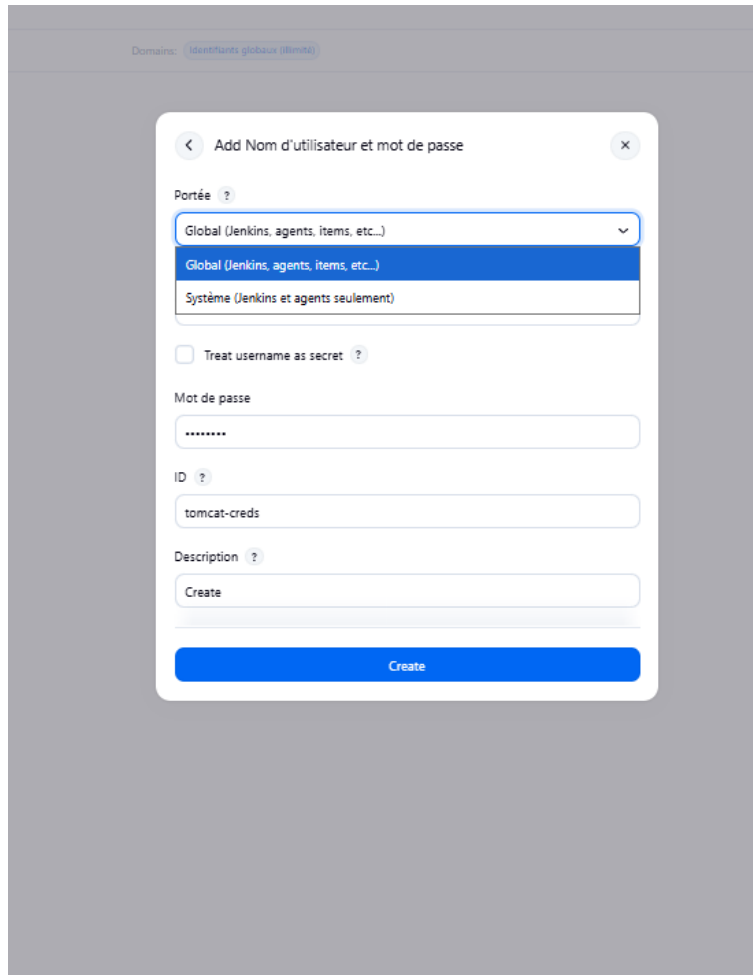
Demarche

J'ai fait evoluer le pom.xml : passage du packaging en war, ajout du plugin maven-war-plugin et creation d'une page src/main/webapp/index.jsp qui affiche les resultats de la classe Calculator (addition, soustraction, multiplication, division), afin d'avoir une preuve visuelle du deploiement dans le navigateur.

J'ai demarre un conteneur Tomcat sur le meme reseau Docker, avec un utilisateur manager configure dans tomcat-users.xml (credentials declares dans Jenkins sous l'ID tomcat-creds) et la valve de restriction d'IP par default (RemoteAddrValve) retiree du context.xml, car elle empechait Jenkins (qui tourne dans un conteneur different, donc pas en localhost du point de vue de Tomcat) d'accéder au Manager.



Tomcat démarre, avant tout déploiement (404 attendu sur la racine)



Domains: Identifiants globaux (global)

< Add Nom d'utilisateur et mot de passe ×

Portée ?

Global (Jenkins, agents, items, etc...)

Global (Jenkins, agents, items, etc...)

Système (Jenkins et agents seulement)

☐ Treat username as secret ?

Mot de passe

ID ?

tomcat-creds

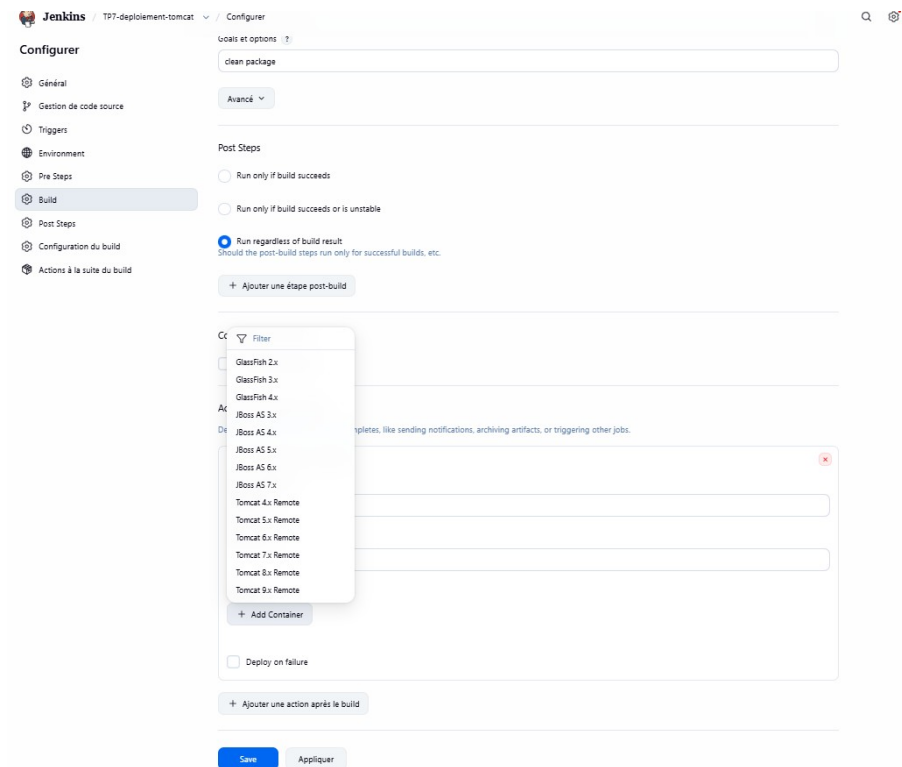
Description ?

Create

Create

Ajout du credential tomcat-creds dans Jenkins

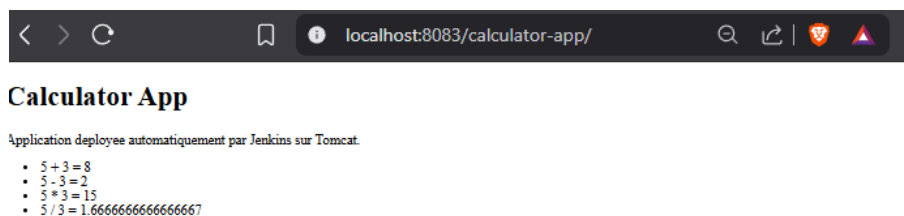
Pour l'étape de déploiement, j'ai utilisé le plugin Jenkins «Deploy to container». En configurant le conteneur cible, j'ai remarqué que la liste déroulante des versions Tomcat supportées s'arrêtait à «Tomcat 9.x Remote» : le plugin ne gère pas Tomcat 10.x. J'ai donc remplacé mon image Tomcat 10 par une image tomcat:9-jdk17-temurin, et refait la configuration du Manager sur cette version.



Liste des versions Tomcat supportees par le plugin de deployment (max 9.x)

Resultats obtenus

Après avoir corrigé la version de Tomcat, le job Jenkins a bien construit le fichier WAR puis l'a déployé automatiquement sur le Tomcat 9. En ouvrant l'application dans le navigateur, la page JSP affichait correctement les résultats calculés par la classe Calculator, confirmant que le déploiement était fonctionnel de bout en bout.



Application calculator-app deployee et accessible sur Tomcat

J'ai ensuite modifié une ligne du fichier index.jsp et re-pousse le commit : grâce au webhook déjà en place, Jenkins a automatiquement relancé le build, reconstruit le WAR et redéploie l'application, ce qui montre que la chaîne complète (push -> build -> déploiement) fonctionne sans intervention manuelle.

Livrable

LIVRABLE

Application packagee en WAR et deployee automatiquement sur Tomcat par Jenkins, avec redeploiement automatique verifie via le webhook GitHub.

TP7 - Jenkins Pipeline (Jenkinsfile)

Objectif

Passer d'une suite de jobs Freestyle a un pipeline déclaratif unique, dont la définition est versionnée dans le dépôt Git au lieu d'être uniquement configurée dans l'interface Jenkins.

Démarche

J'ai d'abord testé un pipeline en collant directement un script Groovy dans le champ «Pipeline script» du job, pour vérifier que la syntaxe déclarative fonctionnait (étapes Checkout, Build, Test).

tp-jenkins

Général Build

Description

Triggers

☒ Build périodiquement

☐ Construire après la build sur d'autres pages

☐ Construire périodiquement

☐ GitHub hook trigger for GITScm polling

☐ Sonnet de l'outil de gestion de version

☐ Déclencher les builds à distance (par exemple à partir de scripts)

Pipeline

Définir votre Pipeline using Groovy directly or pull it from source control

Definition

Pipeline script

```

1  git url: 'https://github.com/frebelkacem-cell/devops-jenkins-tp.git', branch: 'main'
2  }
3  }
4  }
5  }
6  }
7  }
8  }
9  }
10 }
11 }
12 }
13 }
14 }
15 }
16 }
17 }
18 }
19 }
20 }
21 }
22 }
23 }
24 }
25 }
26 }
27 }
28 }
29 }
30 }
31 }
32 }
33 }
34 }
35 }
36 }
37 }
38 }
39 }
40 }
41 }
42 }
43 }
44 }
45 }
46 }
47 }
48 }
49 }
50 }
51 }
52 }
53 }
54 }
55 }
56 }
57 }
58 }
59 }
60 }
61 }
62 }
63 }
64 }
65 }
66 }
67 }
68 }
69 }
70 }
71 }
72 }
73 }
74 }
75 }
76 }
77 }
78 }
79 }
80 }
81 }
82 }
83 }
84 }
85 }
86 }
87 }
88 }
89 }
90 }
91 }
92 }
93 }
94 }
95 }
96 }
97 }
98 }
99 }
100 }
101 }
102 }
103 }
104 }
105 }
106 }
107 }
108 }
109 }
110 }
111 }
112 }
113 }
114 }
115 }
116 }
117 }
118 }
119 }
120 }
121 }
122 }
123 }
124 }
125 }
126 }
127 }
128 }
129 }
130 }
131 }
132 }
133 }
134 }
135 }
136 }
137 }
138 }
139 }
140 }
141 }
142 }
143 }
144 }
145 }
146 }
147 }
148 }
149 }
150 }
151 }
152 }
153 }
154 }
155 }
156 }
157 }
158 }
159 }
160 }
161 }
162 }
163 }
164 }
165 }
166 }
167 }
168 }
169 }
170 }
171 }
172 }
173 }
174 }
175 }
176 }
177 }
178 }
179 }
180 }
181 }
182 }
183 }
184 }
185 }
186 }
187 }
188 }
189 }
190 }
191 }
192 }
193 }
194 }
195 }
196 }
197 }
198 }
199 }
200 }
201 }
202 }
203 }
204 }
205 }
206 }
207 }
208 }
209 }
210 }
211 }
212 }
213 }
214 }
215 }
216 }
217 }
218 }
219 }
220 }
221 }
222 }
223 }
224 }
225 }
226 }
227 }
228 }
229 }
230 }
231 }
232 }
233 }
234 }
235 }
236 }
237 }
238 }
239 }
240 }
241 }
242 }
243 }
244 }
245 }
246 }
247 }
248 }
249 }
250 }
251 }
252 }
253 }
254 }
255 }
256 }
257 }
258 }
259 }
260 }
261 }
262 }
263 }
264 }
265 }
266 }
267 }
268 }
269 }
270 }
271 }
272 }
273 }
274 }
275 }
276 }
277 }
278 }
279 }
280 }
281 }
282 }
283 }
284 }
285 }
286 }
287 }
288 }
289 }
290 }
291 }
292 }
293 }
294 }
295 }
296 }
297 }
298 }
299 }
300 }
301 }
302 }
303 }
304 }
305 }
306 }
307 }
308 }
309 }
310 }
311 }
312 }
313 }
314 }
315 }
316 }
317 }
318 }
319 }
320 }
321 }
322 }
323 }
324 }
325 }
326 }
327 }
328 }
329 }
330 }
331 }
332 }
333 }
334 }
335 }
336 }
337 }
338 }
339 }
340 }
341 }
342 }
343 }
344 }
345 }
346 }
347 }
348 }
349 }
350 }
351 }
352 }
353 }
354 }
355 }
356 }
357 }
358 }
359 }
360 }
361 }
362 }
363 }
364 }
365 }
366 }
367 }
368 }
369 }
370 }
371 }
372 }
373 }
374 }
375 }
376 }
377 }
378 }
379 }
380 }
381 }
382 }
383 }
384 }
385 }
386 }
387 }
388 }
389 }
390 }
391 }
392 }
393 }
394 }
395 }
396 }
397 }
398 }
399 }
400 }
401 }
402 }
403 }
404 }
405 }
406 }
407 }
408 }
409 }
410 }
411 }
412 }
413 }
414 }
415 }
416 }
417 }
418 }
419 }
420 }
421 }
422 }
423 }
424 }
425 }
426 }
427 }
428 }
429 }
430 }
431 }
432 }
433 }
434 }
435 }
436 }
437 }
438 }
439 }
440 }
441 }
442 }
443 }
444 }
445 }
446 }
447 }
448 }
449 }
450 }
451 }
452 }
453 }
454 }
455 }
456 }
457 }
458 }
459 }
460 }
461 }
462 }
463 }
464 }
465 }
466 }
467 }
468 }
469 }
470 }
471 }
472 }
473 }
474 }
475 }
476 }
477 }
478 }
479 }
480 }
481 }
482 }
483 }
484 }
485 }
486 }
487 }
488 }
489 }
490 }
491 }
492 }
493 }
494 }
495 }
496 }
497 }
498 }
499 }
500 }
501 }
502 }
503 }
504 }
505 }
506 }
507 }
508 }
509 }
510 }
511 }
512 }
513 }
514 }
515 }
516 }
517 }
518 }
519 }
520 }
521 }
522 }
523 }
524 }
525 }
526 }
527 }
528 }
529 }
530 }
531 }
532 }
533 }
534 }
535 }
536 }
537 }
538 }
539 }
540 }
541 }
542 }
543 }
544 }
545 }
546 }
547 }
548 }
549 }
550 }
551 }
552 }
553 }
554 }
555 }
556 }
557 }
558 }
559 }
560 }
561 }
562 }
563 }
564 }
565 }
566 }
567 }
568 }
569 }
570 }
571 }
572 }
573 }
574 }
575 }
576 }
577 }
578 }
579 }
580 }
581 }
582 }
583 }
584 }
585 }
586 }
587 }
588 }
589 }
590 }
591 }
592 }
593 }
594 }
595 }
596 }
597 }
598 }
599 }
600 }
601 }
602 }
603 }
604 }
605 }
606 }
607 }
608 }
609 }
610 }
611 }
612 }
613 }
614 }
615 }
616 }
617 }
618 }
619 }
620 }
621 }
622 }
623 }
624 }
625 }
626 }
627 }
628 }
629 }
630 }
631 }
632 }
633 }
634 }
635 }
636 }
637 }
638 }
639 }
640 }
641 }
642 }
643 }
644 }
645 }
646 }
647 }
648 }
649 }
650 }
651 }
652 }
653 }
654 }
655 }
656 }
657 }
658 }
659 }
660 }
661 }
662 }
663 }
664 }
665 }
666 }
667 }
668 }
669 }
670 }
671 }
672 }
673 }
674 }
675 }
676 }
677 }
678 }
679 }
680 }
681 }
682 }
683 }
684 }
685 }
686 }
687 }
688 }
689 }
690 }
691 }
692 }
693 }
694 }
695 }
696 }
697 }
698 }
699 }
700 }
701 }
702 }
703 }
704 }
705 }
706 }
707 }
708 }
709 }
710 }
711 }
712 }
713 }
714 }
715 }
716 }
717 }
718 }
719 }
720 }
721 }
722 }
723 }
724 }
725 }
726 }
727 }
728 }
729 }
730 }
731 }
732 }
733 }
734 }
735 }
736 }
737 }
738 }
739 }
740 }
741 }
742 }
743 }
744 }
745 }
746 }
747 }
748 }
749 }
750 }
751 }
752 }
753 }
754 }
755 }
756 }
757 }
758 }
759 }
760 }
761 }
762 }
763 }
764 }
765 }
766 }
767 }
768 }
769 }
770 }
771 }
772 }
773 }
774 }
775 }
776 }
777 }
778 }
779 }
780 }
781 }
782 }
783 }
784 }
785 }
786 }
787 }
788 }
789 }
790 }
791 }
792 }
793 }
794 }
795 }
796 }
797 }
798 }
799 }
800 }
801 }
802 }
803 }
804 }
805 }
806 }
807 }
808 }
809 }
810 }
811 }
812 }
813 }
814 }
815 }
816 }
817 }
818 }
819 }
820 }
821 }
822 }
823 }
824 }
825 }
826 }
827 }
828 }
829 }
830 }
831 }
832 }
833 }
834 }
835 }
836 }
837 }
838 }
839 }
840 }
841 }
842 }
843 }
844 }
845 }
846 }
847 }
848 }
849 }
850 }
851 }
852 }
853 }
854 }
855 }
856 }
857 }
858 }
859 }
860 }
861 }
862 }
863 }
864 }
865 }
866 }
867 }
868 }
869 }
870 }
871 }
872 }
873 }
874 }
875 }
876 }
877 }
878 }
879 }
880 }
881 }
882 }
883 }
884 }
885 }
886 }
887 }
888 }
889 }
890 }
891 }
892 }
893 }
894 }
895 }
896 }
897 }
898 }
899 }
900 }
901 }
902 }
903 }
904 }
905 }
906 }
907 }
908 }
909 }
910 }
911 }
912 }
913 }
914 }
915 }
916 }
917 }
918 }
919 }
920 }
921 }
922 }
923 }
924 }
925 }
926 }
927 }
928 }
929 }
930 }
931 }
932 }
933 }
934 }
935 }
936 }
937 }
938 }
939 }
940 }
941 }
942 }
943 }
944 }
945 }
946 }
947 }
948 }
949 }
950 }
951 }
952 }
953 }
954 }
955 }
956 }
957 }
958 }
959 }
960 }
961 }
962 }
963 }
964 }
965 }
966 }
967 }
968 }
969 }
970 }
971 }
972 }
973 }
974 }
975 }
976 }
977 }
978 }
979 }
980 }
981 }
982 }
983 }
984 }
985 }
986 }
987 }
988 }
989 }
990 }
991 }
992 }
993 }
994 }
995 }
996 }
997 }
998 }
999 }
1000 }

```

☒ Use Groovy Sandbox

Pipeline Syntax

Advanced

Avancé

Save Appliquer

Premier test : script de pipeline colle directement dans la configuration du job

Une fois la logique validée, j'ai créé un fichier Jenkinsfile à la racine du dépôt GitHub et reconfiguré le job pour utiliser «Pipeline script from SCM», en pointant vers ce même dépôt (branche main). Le Jenkinsfile définit trois étapes principales, plus une étape de résumé :

```

pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps {
                git url: 'https://github.com/frebelkacem-cell/devops-jenkins-tp.git', branch:
'main'
            }
        }
    }
}

```

```
stage('Build') {
    steps {
        sh 'mvn clean package'
    }
}
stage('Test') {
    steps {
        junit 'target/surefire-reports/*.xml'
    }
}
stage('Summary') {
    steps {
        echo 'Pipeline mis a jour automatiquement via webhook GitHub.'
    }
}
}
```

Resultats obtenus

Le pipeline s'exécute désormais automatiquement à chaque push (grâce au webhook déjà configuré), en enchaînant checkout, build Maven, publication des tests JUnit puis un message récapitulatif dans les logs. L'avantage par rapport aux jobs Freestyle précédents est que toute modification du pipeline (ajout d'une étape, changement d'ordre...) passe désormais par un commit Git, donc par un historique et une revue possible, plutôt que par un simple clic dans l'interface.

Livrable

LIVRABLE

Jenkinsfile déclaratif versionné dans GitHub, exécuté automatiquement par Jenkins via «Pipeline script from SCM», avec build, tests et étape de résumé.

TP8 - Architecture maitre-esclave (agent Jenkins)

Objectif

Ajouter un agent (noeud esclave) a l'instance Jenkins, afin de repartir l'execution des builds entre le controleur et cet agent.

Demarche

Le TP proposait a l'origine un agent connecte en SSH (image jenkins/ssh-agent). J'ai plutot choisi un agent en connexion entrante (image jenkins/inbound-agent, protocole JNLP/WebSocket), plus simple a mettre en place avec des conteneurs Docker sur le meme reseau devops-net, sans avoir a gerer de cles SSH supplementaires.

Dans Jenkins, j'ai cree un nouveau noeud (agent1), avec un seul executeur, un repertoire de travail /home/jenkins/agent, une etiquette linux (pour pouvoir cibler ce noeud depuis un job ou un pipeline), et comme methode de lancement «Launch agent by connecting it to the controller».

The screenshot shows the Jenkins 'New Node' configuration page. The form is titled 'Description' and contains several fields: 'Number of executors' set to 1, 'Remote directory' set to /home/jenkins/agent, 'Labels' set to linux, 'Usage' set to 'Utiliser ce noeud autant que possible', 'Launch method' set to 'Launch agent by connecting it to the controller', and 'Availability' set to 'Maintenir l'agent active le plus longtemps possible'. There are also checkboxes for 'Disable deferred wipeout on this node', 'Disk Space Monitoring Thresholds', 'Emplacement des outils', and 'Variables d'environnement'. A blue 'Enregistrer' button is at the bottom.

Configuration du noeud agent1 dans Jenkins

Jenkins m'a alors fourni la commande de connexion (avec un secret d'authentification, masque sur la capture ci-dessous) a executer dans le conteneur agent pour qu'il se connecte au controleur.

Agent agent1

Run from agent command line: (Unix)

[secret masqué]

Run from agent command line: (Windows)

[secret masqué]

Or run from agent command line with the secret stored in a file: (Unix)

[secret masqué]

```
java -jar agent.jar -url http://localhost:8081/ -secret @secret-file -name agent1 -websocket -workdir "/home/jenkins/agent"
```

Or run from agent command line with the secret stored in a file: (Windows)

[secret masqué]

```
java -jar agent.jar -url http://localhost:8081/ -secret @secret-file -name agent1 -websocket -workdir "/home/jenkins/agent"
```

If you prefer to use TCP instead of WebSockets, remove the -websocket option. Run `java -jar agent.jar -help` for more.

Note: PowerShell users must use `curl.exe` instead of `curl` because `curl` is a default PowerShell cmdlet alias for `Invoke-WebRequest`.

Monitoring Data

Labels

linux

Projets rattachés à agent1

Aucun

Commandes de connexion fournies par Jenkins pour l'agent1 (secret masqué)

Resultats obtenus

Une fois le conteneur agent démarre avec cette commande, agent1 est apparu connecte dans Jenkins. En lançant plusieurs jobs (dont un explicitement affecte au noeud agent1 via son etiquette), j'ai pu verifier sur le tableau de bord que les builds se repartissaient bien entre le controleur et l'agent, chacun avec sa propre file d'attente de compilation.

S	M	Nom du projet	Dernier succès	Dernier échec	Dernière durée
🔄	☀️	TP1-compilation	s. o.	s. o.	ND
✅	☀️	TP1-compilation belkacem	50 s (#5)	1 j 3 h (#2)	5.8 s
✅	☀️	TP2-maven-project	50 mn (#10)	1 j 0 h (#2)	33 s
✅	☀️	TP7-deploiement-tomcat	50 mn (#6)	s. o.	22 s
✅	☀️	TP8-pipeline-dsl	50 mn (#4)	59 mn (#1)	32 s
✅	☀️	TP9-repartition-charge	1 mn 16 s (#1)	s. o.	26 s

Tableau de bord Jenkins : jobs repartis entre le controleur et l'agent1

Livrable

LIVRABLE

Agent Jenkins connecte et operationnel, avec des builds effectivement executes a la fois sur le controleur et sur l'agent, selon l'etiquette du job.

TP9 - Securite, sauvegarde et restauration

Objectif

Mettre en place une gestion des droits par utilisateur dans Jenkins, puis verifier qu'il est possible de sauvegarder et de restaurer une instance Jenkins sans perte de configuration.

Demarche - Securite

Dans Administrer Jenkins > Security, j'ai remplace l'autorisation par default («Les utilisateurs connectes peuvent tout faire») par une «Strategie d'autorisation matricielle basee sur les projets», qui permet de definir precisement, pour chaque utilisateur, les droits accordes (administration, lecture, build, gestion des identifiants...).

Jenkins / Administrer Jenkins / Security

Security

Sécuriser Jenkins; définir qui est autorisé à accéder au système.

Authentication

☐ Disable "Keep me signed in" ?

Royaume pour la sécurité (Realm)

Base de données des utilisateurs de Jenkins

☐ Autoriser les utilisateurs à s'inscrire ?

Autorisations

Les utilisateurs connectés peuvent tout faire

Les utilisateurs connectés peuvent tout faire

Mode legacy

Stratégie d'autorisation matricielle basée sur les projets

Sécurité basée sur une matrice

Tout le monde a accès à toutes les fonctionnalités

Markup romacter ?

Texte brut

Considère toutes les entrées comme du texte brut. Les caractères HTML, non sécurisés HTML, comme < et & sont échappés en leurs entités HTML respectives.

Choix de la strategie d'autorisation matricielle

☐ Disable "Keep me signed in" ?

Royaume pour la sécurité (Realm)

Base de données des utilisateurs de Jenkins

☐ Autoriser les utilisateurs à s'inscrire ?

Autorisations

Sécurité basée sur une matrice

Utilisateur/groupe	Global			Identifiants		Agent				Job				Historique des builds		Vues		Gestion de version
	Administer	Manage	Read	Configure	Update	View	Configure	Connect	Disconnect	Build	Cancel	Configure	Create	Delete	Discover	Move	Read	Workspaces
Utilisateur/groupe																		
👤 Anonyme																		
👤 Authenticated Users																		

Add user... Add group...

Markup Formatter

Markup Formatter ?

Texte brut

Considère toutes les entrées comme du texte brut. Les caractères HTML non sécurisés HTML comme < et & sont échappés en leurs entités HTML respectives.

Matrice de permissions vide, avant attribution des droits

J'ai ensuite cree trois comptes utilisateurs avec des roles distincts : admin (tous les droits, y compris Administer), dev1 (droits de build et de lecture de l'espace de travail, pour lancer des jobs sans pouvoir changer la configuration globale), et lecteur1 (droit de lecture seule, sans aucun droit de build ni de configuration).

 **Jenkins** / Administrer Jenkins / Security

Security

Sécuriser Jenkins; définir qui est autorisé à accéder au système.

Authentication

☐ Disable "Keep me signed in" ?

Royaume pour la sécurité (Realm)

Base de données des utilisateurs de Jenkins

☐ Autoriser les utilisateurs à s'inscrire ?

Autorisations

Sécurité basée sur une matrice

Utilisateur/groupe	Global			Identifiants		Agent				Job				Historique des builds		Vues		Gestion de version
	Administer	Manage	Read	Configure	Update	View	Configure	Connect	Disconnect	Build	Cancel	Configure	Create	Delete	Discover	Move	Read	Workspaces
Utilisateur/groupe																		
👤 Anonyme																		
👤 Authenticated Users																		
👤 admin	✓																	
👤 dev1										✓							✓	✓
👤 lecteur1		✓															✓	

Add user... Add group...

Markup Formatter

Markup Formatter ?

Matrice finale : droits differencies entre admin, dev1 et lecteur1

J'ai fait attention en configurant cette matrice a bien garder le droit Administer pour mon propre compte avant d'enregistrer, pour eviter le risque classique de me retrouver bloqué hors de l'interface d'administration.

Demarche - Sauvegarde et restauration

Plutôt que d'installer un plugin de sauvegarde, j'ai fait une sauvegarde «a froid» du volume Docker jenkins_home directement, en passant par un conteneur temporaire qui monte ce volume et qui archive son contenu avec tar :

```
docker run --rm --volumes-from jenkins -v ${PWD}:/backup ubuntu tar czf
/backup/jenkins_home_backup.tar.gz /var/jenkins_home
```

Pour verifier que cette sauvegarde etait reellement exploitable (et pas seulement presente sur le disque), je l'ai restauree dans un volume Docker separe, different du volume de production, afin de ne pas risquer d'endommager mon instance Jenkins reelle :

```
docker volume create jenkins_home_test
docker run --rm -v jenkins_home_test:/var/jenkins_home -v ${PWD}:/backup ubuntu
tar xzf /backup/jenkins_home_backup.tar.gz -C /
```

J'ai ensuite demarre un conteneur Jenkins temporaire sur ce volume de test, sur un port different de l'instance principale, et j'ai verifie que le tableau de bord affichait bien tous les jobs, l'historique des builds et la configuration de securite identiques a l'instance d'origine. Une fois la restauration validee, j'ai supprime le conteneur et le volume de test.

Livrable

LIVRABLE

Securite Jenkins basee sur une matrice de droits par utilisateur, et procedure de sauvegarde/restauration du volume Jenkins verifiee dans un environnement de test isole.

Conclusion

Ce cours m'a permis de mettre en pratique, sur un seul projet reutilise de bout en bout, l'ensemble d'une chaine d'integration et de livraison continue avec Jenkins : compilation simple, build Maven avec tests unitaires, declenchement automatique via webhook GitHub, analyse de qualite de code avec SonarQube et JaCoCo, jobs parametres, deploiement automatise sur Tomcat, ecriture d'un pipeline declaratif versionne, ajout d'un agent de build, puis securisation et sauvegarde de l'instance.

Le fait de garder le meme projet calculator-app d'un TP a l'autre m'a permis de mieux comprendre comment ces briques s'assemblent concretement dans un vrai pipeline, plutot que de les voir comme des exercices isoless. Les quelques erreurs rencontrees en cours de route (bug d'encodage sur un commit, oubli de push avant une analyse SonarQube, incompatibilite de version avec le plugin de deploiement Tomcat) ont finalement ete aussi formatrices que les etapes qui ont fonctionne du premier coup, car elles obligent a lire et comprendre les messages d'erreur de Jenkins plutot que de suivre une procedure sans se poser de questions.